

เอกสารประกอบโปรเจค

ssdeep-scanner

CLI-based Heuristic Malware Scanner (Go Edition)

1. บทนำและวัตถุประสงค์

ssdeep-scanner - เป็นโปรแกรม Command Line ที่เขียนด้วยภาษา Go version 1.25 ขึ้นไปเพื่อช่วยค้นหาไฟล์ที่มีความคล้ายกับตัวอย่างมัลแวร์จากฐานข้อมูลลายเซ็นโดย ssdeep เป็น fuzzy hashing ที่เหมาะกับการเปรียบเทียบไฟล์ที่ไม่เหมือนกันแบบ byte-to-byte แต่ยังมีโครงสร้างหรือเนื้อหาคล้ายกัน เป้าหมายของโปรเจกต์ไม่ใช่การแทนที่ Antivirus แบบเต็มรูปแบบ แต่เป็นเครื่องมือช่วยวิเคราะห์เชิง heuristic สำหรับค้นหาไฟล์น่าสงสัยในโฟลเดอร์หรือทั้งระบบ โดยมี layer เสริมเพื่อช่วยลด false positive

Use case ที่เหมาะสม

- สแกนเครื่องหรือโฟลเดอร์เพื่อค้นหาไฟล์ executable/script ที่คล้ายกับมัลแวร์ที่มีลายเซ็นอยู่แล้ว
- ใช้กับงาน incident response, malware triage, lab analysis หรือ endpoint quick check
- นำเข้าฐานลายเซ็นจากไฟล์ JSON/SQL/CSV หรือดึง ssdeep จาก VirusTotal ด้วย hash
- ใช้ cache ลดเวลาการสแกนซ้ำเมื่อไฟล์ไม่ได้เปลี่ยนแปลง
- ใช้ signature โหลดเข้าหน่วยความจำเพื่อเพิ่มความเร็วจากการโหลดล่วงหน้า
- กำหนดนามสกุลไฟล์ที่ต้องการสแกนเพื่อใช้ลดเวลาการสแกนหรือสามารถสแกนรายละเอียดทั้งหมด
- Export ไฟล์ได้ทั้ง CSV/JSON เพื่อรายงานผลที่ได้จากการสแกน

2. โครงสร้างโปรเจค

ไฟล์	บทบาท
main.go	Entry point, โหลด VT keys/trusted publishers, parse CLI flags และแสดง interactive menu
scanner.go	แกนหลักของการสแกน: file discovery, pre-filter, cache, worker pool, ssdeep comparison, alert/warn/suppress/clean logic
db.go	สร้างและเชื่อมต่อ SQLite database สำหรับ signatures.db และ cache.db รวมถึง update cache
import.go	นำเข้าลายเซ็นจาก JSON, SQL, CSV และเพิ่ม signature แบบ manual
export.go	ส่งออกไฟล์ JSON, CSV โดยมี sha256, file_path, ระดับการแจ้งเตือน, ssdeep
vt.go	เชื่อมต่อ VirusTotal API v3, หมุน API keys, คำนวณ SHA-256 และดึง ssdeep จาก VT
malware_model.bin	ส่วนโมเดลที่ใช้กับประเมินไฟล์ว่าโอกาสที่จะเป็นมัลแวร์เป็น 0.587 ขึ้นไป
signatures.db	ฐานข้อมูลมัลแวร์ที่ใช้ในการเปรียบเทียบกับไฟล์ที่จะตรวจสอบว่าคล้ายคลึงเท่าไร
trusted.go	โหลดรายชื่อ trusted publishers จาก embedded file และ override จาก trusted_publishers.txt บนดิสก์
trusted_windows.go	Windows-only: ตรวจสอบ Authenticode ด้วย WinVerifyTrust และอ่าน CompanyName จาก PE Version Info
trusted_other.go	Non-Windows stub เพื่อให้ build บน Linux/macOS ได้
trusted_publishers.txt	รายชื่อ publisher ที่เชื่อถือได้ ใช้กับ CompanyName partial match
README.md	คู่มือใช้งานพื้นฐานของโปรเจค

3. Dependencies และเหตุผลที่ใช้

Dependency	หน้าที่
github.com/glaslos/ssdeep	คำนวณ ssdeep fuzzy hash และวัด similarity score
github.com/dmitryikh/leaves	โมเดล XGBOOST สำหรับคัดกรอง False Positive/Negative
golang.org/x/sys/windows	เรียก Windows API สำหรับ Authenticode และ Version Info
modernc.org/sqlite	SQLite driver แบบ pure Go ไม่ต้องพึ่ง CGO เหมาะกับ cross-platform build

4. ภาพรวมการทำงานระดับสูง

```

Start
-> Load VirusTotal keys from vt_keys.txt (optional)
-> Load trusted publishers
-> Parse CLI / interactive menu
-> Initialize SQLite databases
-> Load signature into memory
-> Load scan cache into memory
-> Walk target directories
-> Filter by extension, size, excluded dirs, cache
-> Send file jobs to worker pool
-> Compute ssdeep hash
-> Query signatures by block size
-> Compare similarity score
-> Apply Authenticode / Trusted Publisher / ML Filters / VT suppression
-> Print ALERT/WARN/CLEAN and update cache
-> Print scan summary

```

การทำงานแบ่งเป็น 2 phase หลัก: phase discovery ที่เดินไฟล์และกรองไฟล์เบื้องต้น และ phase worker ที่คำนวณ hash เปรียบเทียบลายเซ็น และตัดสินผลลัพธ์

5. ขั้นตอนสแกนแบบละเอียด

5.1 ตรวจ target directory

ฟังก์ชัน ScanTargets รับ list ของ directory แล้วตรวจด้วย os.Stat หากไม่มี directory ที่ถูกต้องจะจบโปรแกรมด้วย error

5.2 Initial database

initDatabases สร้าง signatures.db และ cache.db หากยังไม่มี โดย signatures.db เก็บลายเซ็น ssdeep ส่วน cache.db เก็บ filepath และ mtime ของไฟล์ที่สแกนแล้ว

5.3 โหลด cache เข้า memory

loadCache อ่าน scan_cache ทั้งหมดมาเก็บใน map เพื่อ lookup เร็วระหว่างเดินไฟล์ ถ้า filepath และ mtime ตรงกันจะข้ามการสแกนไฟล์นั้น

5.4 โหลด signature เข้า memory

ดึงข้อมูลลายเซ็นมัลแวร์จากฐานข้อมูลมาเตรียมไว้ในหน่วยความจำ (Pre-loading) เพื่อนำไปใช้เปรียบเทียบในภายหลัง ซึ่งการโหลดข้อมูลมาเก็บไว้ล่วงหน้าจะช่วยให้ระบบทำงานและค้นหาได้อย่างรวดเร็ว

5.5 เดินไฟล์ด้วย filepath.WalkDir

โปรแกรมเดินทุก target directory และใช้ isExcludedDir เพื่อข้าม path ที่ไม่ควรสแกน เช่น Windows\WinSxS, \$Recycle.Bin, /proc, /sys, /dev, /run

5.6 Pre-filter ssdeep-scanner

ไฟล์ต้องเป็น regular file, extension อยู่ใน TargetExtensions และขนาดอยู่ระหว่าง 4KB ถึง 50MB เท่านั้น จึงจะถูกส่งเข้า worker pool

5.7 Worker pool

จำนวน worker เท่ากับ `runtime.NumCPU()` เพื่อใช้ CPU หลาย core ในการคำนวณ ssdeep และเปรียบเทียบฐานข้อมูล โดยส่งงานผ่าน channel jobs

5.8 ssdeep และ block size optimization

หลังคำนวณ ssdeep ของไฟล์ โปรแกรมดึง block size จาก hash แล้ว query signature เฉพาะ `block_size` ที่สัมพันธ์กัน คือ `blockSize/2`, `blockSize` และ `blockSize*2` เพื่อลดจำนวน candidate ที่ต้องเปรียบเทียบ

5.9 Similarity threshold Threshold

พื้นฐานถูกตั้งเป็น 85 หมายถึงถ้า `ssdeep.Distance` ระหว่างไฟล์และ signature ได้คะแนนตั้งแต่ 85 ขึ้นไป จึงถือว่า match และเข้าสู่ขั้นตอนลด false positive สามารถเปลี่ยนค่า Threshold ได้

5.10 Target Extension Files

สำหรับคัดกรองไฟล์ที่ต้องการหรือเลือกสแกนนามสกุลไฟล์ทั้งหมดด้วย `-all` ช่วยให้เพิ่มความเร็วในการสแกนผลลัพธ์หรือเพื่อสแกนรายละเอียดทั้งหมด

5.11 ML filter (XGBoost)

เป็นการใช้ Machine Learning ทำหน้าที่เป็น "ด่านตรวจคัดกรองเร็ว (Fast-path)" เพื่อแยกไฟล์ปลอดภัยออกจาก การตรวจสแกนเชิงลึก ช่วยลดการใช้ทรัพยากร CPU และลดเวลาในการสแกน นำผลการทำนาย (prediction) มาเปรียบเทียบกับค่าเกณฑ์ความปลอดภัยที่กำหนด หากคะแนนต่ำกว่าเกณฑ์ แสดงว่าโมเดลมั่นใจว่าไฟล์นี้เป็นไฟล์ปกติ/ปลอดภัย (Clean File)

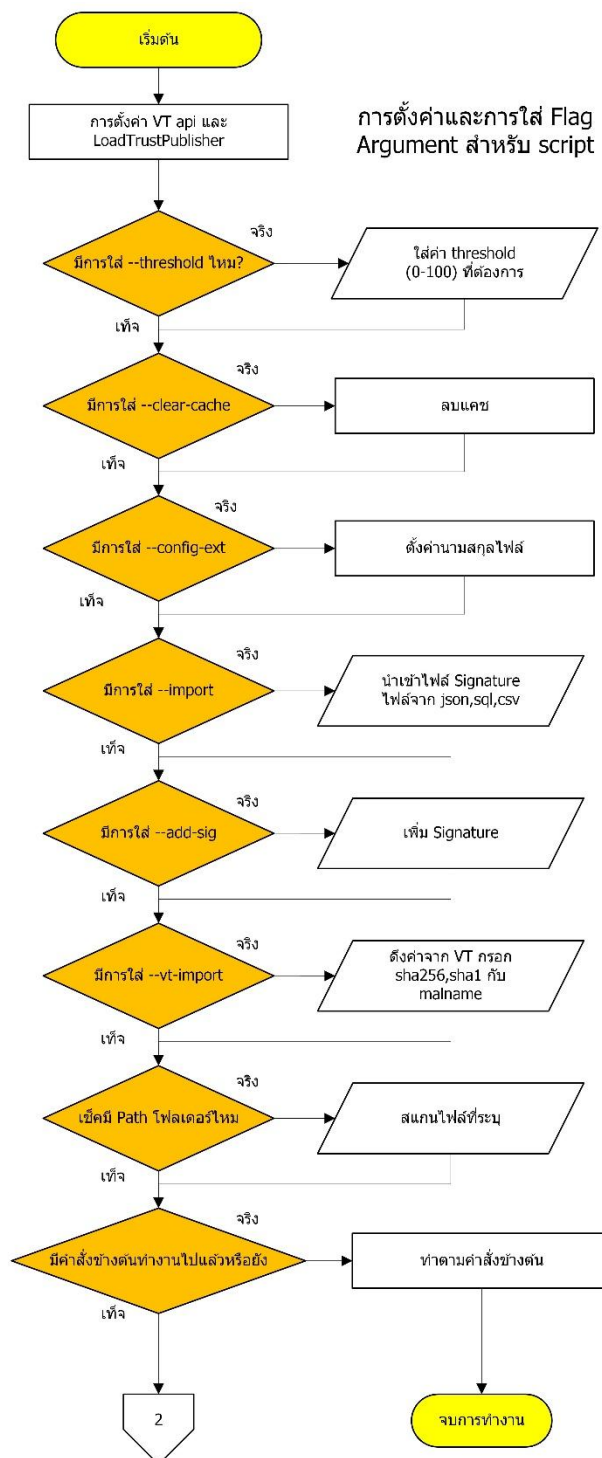
5.12 EXPORT CSV/JSON

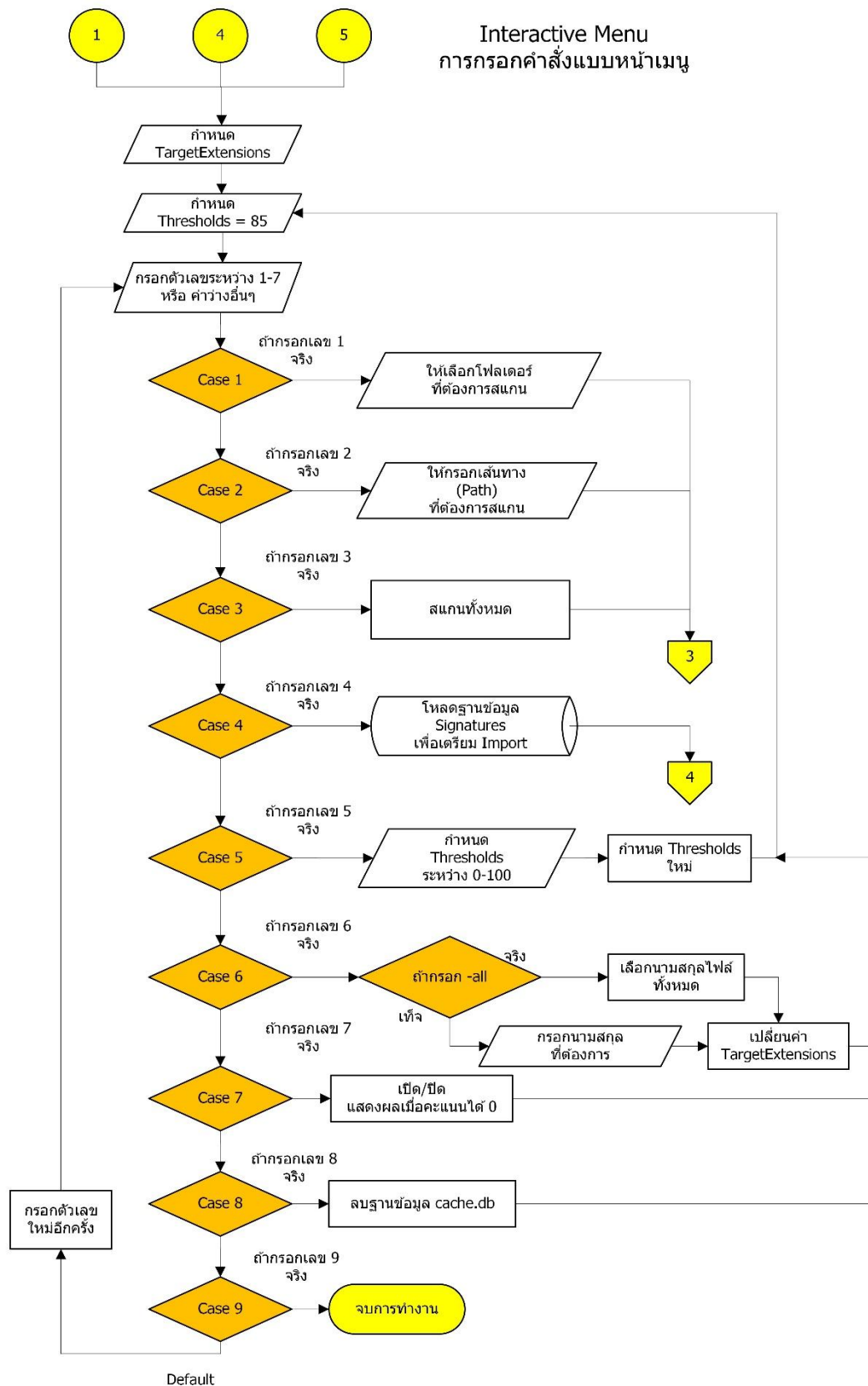
แสดงเมนูตัวเลือกสรุปจำนวนรายการที่พบแจ้งให้ผู้ใช้ทราบ พร้อมแสดงเมนูกด

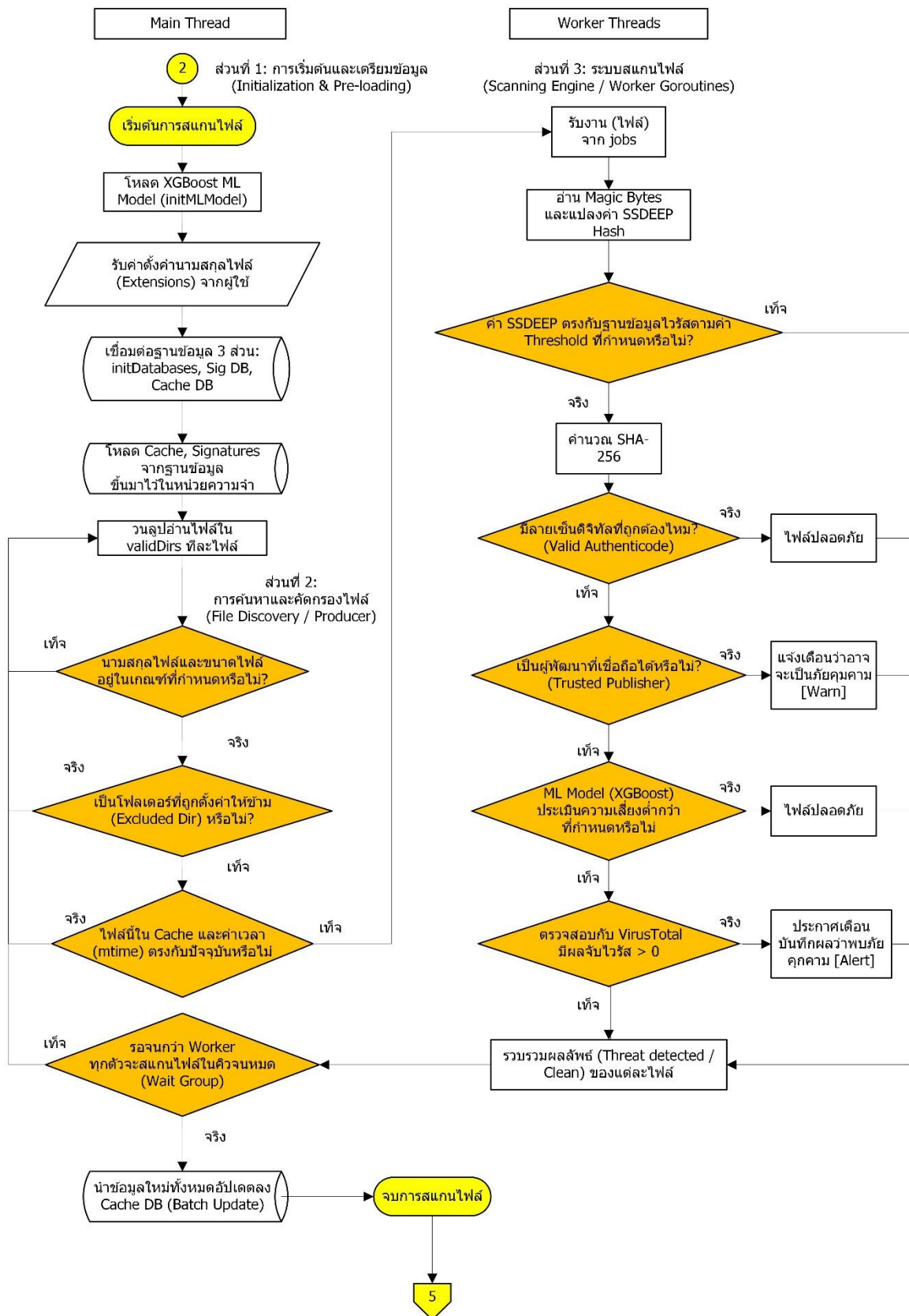
- กด [1]: ส่งออกข้อมูลเป็นไฟล์ JSON (`scan_report.json`)
- กด [2]: ส่งออกข้อมูลเป็นไฟล์ CSV (`scan_report.csv`)
- กด [3]: ข้ามขั้นตอนการส่งออกรายงาน

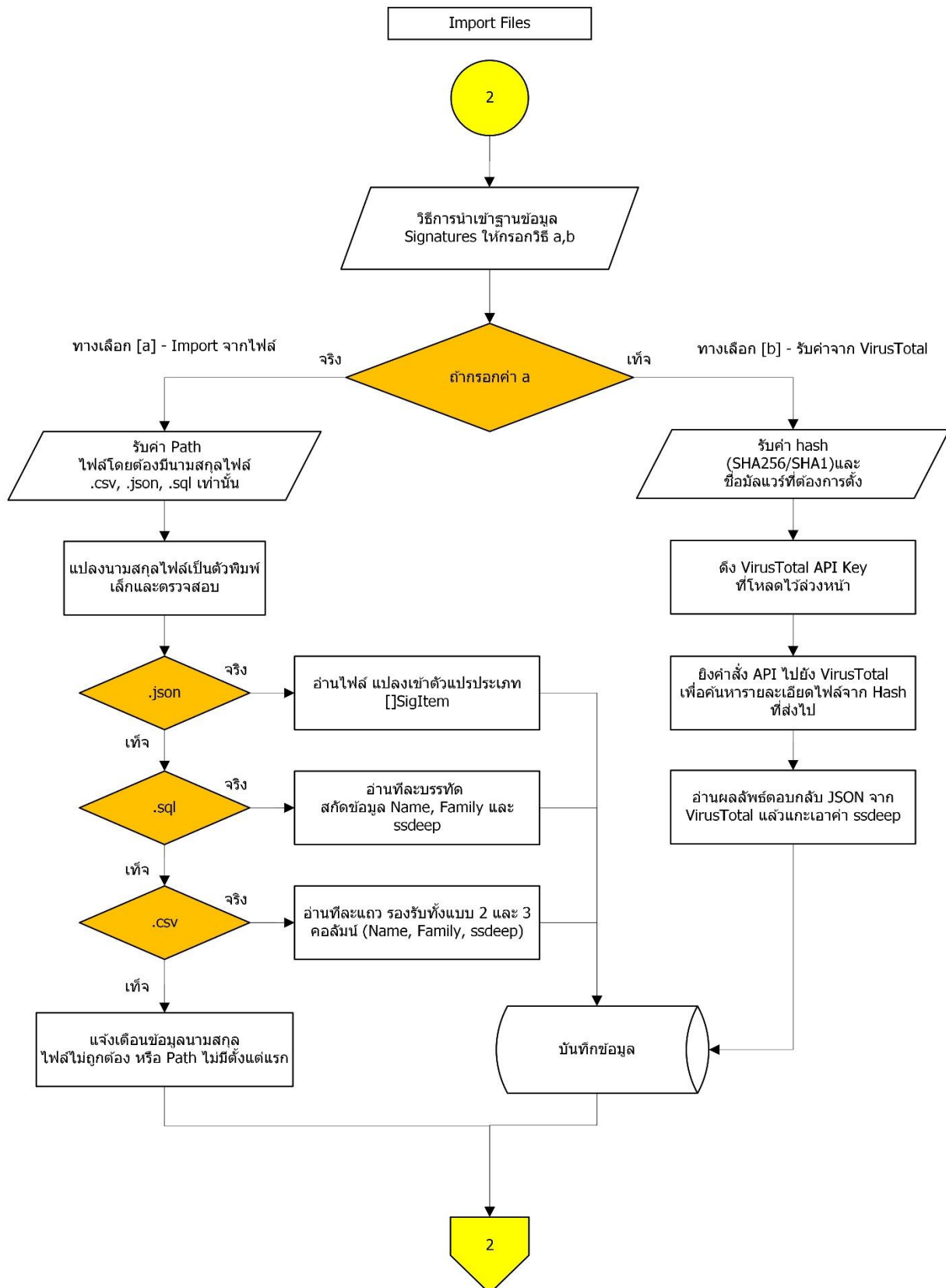
6. Flowchart

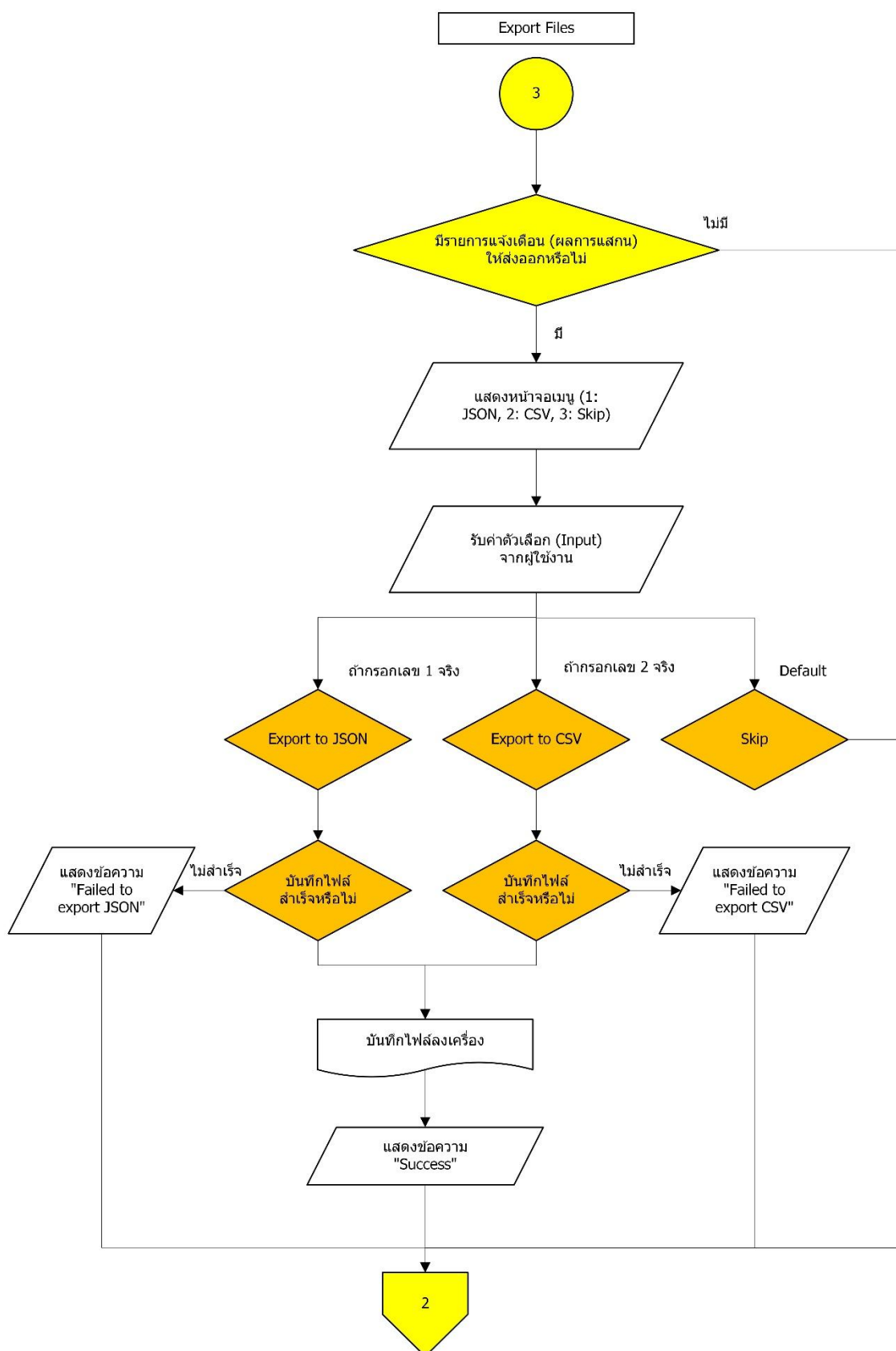
แผนผังการทำงาน (Flowchart) นี้แสดงลำดับขั้นตอนการทำงานของโปรเจกต์ทั้งหมด ซึ่งทำหน้าที่แสดงขั้นตอนและลำดับการทำงานของกระบวนการ โดยใช้สัญลักษณ์มาตรฐานแทนการทำงานแต่ละขั้นตอนดังต่อไปนี้











7. Alert/Warn/Suppress/Clean Logic

ลำดับการตัดสินใจหลัง ssdeep match
1) Authenticode valid: suppress เจียบและนับเป็น SkippedSigned
2) CompanyName ตรง trusted publisher: แสดง [WARN] ไม่ใช่ [ALERT] และนับเป็น SkippedTrusted
3) VirusTotal malicious+suspicious = 0: suppress และนับเป็น SkippedVTClean
4) ถ้าไม่ถูก suppress: แสดง [CLEAN] พร้อมชื่อไฟล์ path score family และ VT/publisher info ถ้ามี

แนวคิดนี้ช่วยลด false positive จากไฟล์ legitimate ที่มี ssdeep คล้ายกับ signature บางตัว แต่ยังคงแสดง warning สำหรับกรณี publisher trusted แบบ unverified เพราะ CompanyName เป็น metadata ที่สามารถปลอมได้

กรณี	ผลลัพธ์	เหตุผล
ไฟล์มี Authenticode valid	Suppress	ลายเซ็นดิจิทัลถูกตรวจสอบผ่าน Windows trust provider
CompanyName match trusted list แต่ไม่มี valid signature	Warn	เป็นสัญญาณลดความรุนแรง แต่ยังไม่ควรเชื่อถือเต็มที่
VT บอก 0 det	Suppress	VT ช่วยยืนยันว่าไม่พบความเสี่ยง ณ เวลาตรวจ
VT บอก 0 det (No suppress)	Clean	แสดง VT ช่วยยืนยันว่าไม่พบความเสี่ยง ณ เวลาตรวจ
ssdeep match และไม่เข้าเงื่อนไข false positive	ALERT	ถือว่าเป็นไฟล์น่าสงสัย

8. ฐานข้อมูล

8.1 signatures.db

```
CREATE TABLE IF NOT EXISTS signatures (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    malware_name TEXT,
    block_size INTEGER,
    ssdeep_full TEXT
);
CREATE INDEX IF NOT EXISTS idx_block_size ON signatures(block_size);
CREATE INDEX IF NOT EXISTS idx_ssdeep_full ON signatures(ssdeep_full);
```

ฐานข้อมูลนี้เป็นหัวใจของการตรวจจับ โดย malware_name คือชื่อมัลแวร์หรือ family, block_size ใช้ optimize query และ ssdeep_full คือ hash เต็มรูปแบบ เช่น block:hash:hash2

8.2 cache.db

```
CREATE TABLE IF NOT EXISTS scan_cache (
    filepath TEXT PRIMARY KEY,
    mtime INTEGER NOT NULL,
    is_threat INTEGER NOT NULL,
    message TEXT,
    sha256 TEXT,
    ssdeep TEXT
);
```

cache ใช้ลดเวลาสแกนซ้ำ หากไฟล์เดิมมี modification time เท่าเดิม โปรแกรมจะข้ามการคำนวณ ssdeep และไม่ query signature ซ้ำ สามารถแจ้งเตือนจากผลลัพธ์ก่อนหน้า

9. การนำเข้าลายเซ็น

โปรแกรมรองรับการเพิ่ม signature หลายช่องทาง เพื่อให้ฐานข้อมูลยืดหยุ่นกับแหล่งข้อมูลหลายรูปแบบ

วิธี	คำอธิบาย
--add-sig	เพิ่มลายเซ็นเดียวผ่าน command line
--import file.json	นำเข้า JSON array ที่มี name/family/ssdeep
--import file.sql	parse INSERT INTO malware_signatures จาก dump SQL
--import file.csv	รองรับ 2 column: name,ssdeep หรือ 3 column: name,family,ssdeep
--vt-import	ดึง ssdeep จาก VirusTotal ด้วย SHA256/SHA1 แล้วเพิ่มเข้า DB

Duplicate handling

importLoop โหลด ssdeep_full ที่มีอยู่แล้วเข้า map และใช้ map เดียวกันกับ duplicate ภายใน batch ด้วย ดังนั้น signature ซ้ำจะถูก skip ไม่ insert ซ้ำ

10. VirusTotal

Integration vt.go มี 2 หน้าหลัก: ตรวจคะแนน VT ของไฟล์ที่ match ระหว่างสแกน และดึง ssdeep จาก VT เพื่อนำเข้า signature database

10.1 การโหลด API keys

LoadVTKeys อ่าน vt_keys.txt แบบ optional หากไม่มีไฟล์จะไม่ error โปรแกรมยังทำงานได้ แต่จะไม่มี VT suppression/score

10.2 Key rotation

getNextVTKey ใช้ mutex ป้องกัน race แล้ววน key แบบ round-robin หาก API response เป็น 429 หรือ 401 จะลอง key ถัดไป

10.3 CheckVTScore

คำนวณ SHA-256 ของไฟล์ แล้วเรียก endpoint /api/v3/files/{sha256} คืนค่า malicious+suspicious และ total engine count

10.4 FetchSsdeepFromVT

เรียก VirusTotal ด้วย hash ที่ผู้ใช้ระบุ หาก response มี attributes.ssdeep จะส่งต่อให้ addSig เพื่อเพิ่ม signature

ข้อควรระวังเรื่องความเป็นส่วนตัว

การเช็ค VT ด้วย hash ไม่อัปโหลดไฟล์ แต่ส่ง SHA-256 ไปยัง VirusTotal ควรระวัง policy องค์กรก่อนใช้ VT กับไฟล์ภายในหรือไฟล์ลับ เอกสารนี้ไม่บันทึกและไม่แสดงค่า API key ใด ๆ

11. Trusted Publisher และ Authenticode

11.1 Authenticode

บน Windows โปรแกรมเรียก WinVerifyTrust ผ่าน wintrust.dll เพื่อตรวจว่าไฟล์มี digital signature ที่ valid หรือไม่ ถ้า valid จะ suppress alert เพราะถือว่าเชื่อถือได้มากกว่า metadata ธรรมดา

11.2 CompanyName

getPECompanyName อ่าน PE Version Info ด้วย version.dll แล้ว query field CompanyName หลาย language/codepage หากพบ CompanyName จะนำไปเทียบกับ trusted publishers

11.3 trusted_publishers.txt

ไฟล์นี้ถูก embed เข้า binary ด้วย //go:embed แต่ LoadTrustedPublishers จะลองเปิด trusted_publishers.txt บนดิสก์ด้วย หากมีรายการ override และไม่ว่าง จะใช้ไฟล์บนดิสก์แทน embedded list การ match เป็น case-insensitive partial match เช่น CompanyName มีคำว่า microsoft corporation จะตรงกับรายการ microsoft corporation

12. วิธีใช้งาน

12.1 Build

```
# Windows
go build -o scanner.exe .
.\scanner.exe
# ติดตั้ง dependencies สำหรับ GUI Popup (ทางเลือก)
sudo apt install zenity kdialog
# Build และกำหนดสิทธิ์
go build -o scanner_linux .
chmod +x scanner_linux
./scanner_linux
```

12.2 Interactive mode

- [1] Select folder via GUI Popup
- [2] Specify directory path manually
- [3] Scan entire system (All Drives)
- [4] Import Signatures (File / VirusTotal)
- [5] Change Threshold
- [6] Configure Target Extensions
- [7] Toggle Suppress Clean VT [Current: ON]
- [8] Clear Cache Database (cache.db)
- [9] Exit Scanner

12.3 Command line help

```
-clear-cache
    Delete cache database file (cache.db)
-config-ext
    Configure target file extensions
-import string
    Import .json, .sql, or .csv signatures file
-suppress-vt
    Suppress VirusTotal clean results (default: true)
-threshold int
    Set SSDEEP similarity threshold score (0-100) (default -1)
-offline
    Disable VirusTotal and run completely offline
```

12.4 Command line examples

```
# Scan a directory scanner.exe
C:\Users\Alice\Downloads

# Scan multiple directories
scanner.exe C:\Users D:\Tools

# Import signatures
scanner.exe --import signatures.json
scanner.exe --import signatures.csv

# Add one ssdeep signature
scanner.exe --add-sig Example.Malware "12288:abc:def"

# Import ssdeep from VirusTotal
scanner.exe --vt-import Example.Malware

# Delete cache.db
.\scanner.exe -clear-cache

# Configuration Files Extension
.\scanner.exe -config-ext

# Suppression (On/Off)
.\scanner.exe -suppress-vt false C:\

# Threshold setting (-threshold int)
.\scanner.exe -threshold 70

# Example of usage
.\scanner.exe -threshold 70 -config-ext --suppress-vt false C:\      #แบบ 1 directory
.\scanner.exe -config-ext -clear-cache C:\ D:\ -threshold 90      #แบบหลาย directory
```

12.5 Extension config

```
=== Configure Target Extensions ===
Enter EXACT extensions to scan separated by commas (e.g., .exe, .dll).
Type 'all' to scan all predefined script/executable extensions > .exe, .dll (all ก็ได้)
# สามารถกรอกนามสกุลได้ต่อไปนี้
.com .msi .msp .scr .pif .cpl .msc .exe .dll .sys .ps1 .bat .cmd .vbs .vbe .jse .wsf .hta .inf
.lnk .url .docm .xslm .pptm .rtf .sh .py .jar .so .dmg .pkg .command
```

13. Scan Summary ที่โปรแกรมแสดง

Metric	ความหมาย
Total files scanned	จำนวนไฟล์ที่ถูกคำนวณ ssdeep จริง
Files skipped (Size/Ext)	ไฟล์ที่ถูกกรองออกเพราะ extension, size หรืออ่าน info ไม่ได้
Files skipped (Cache)	ไฟล์ที่ข้ามเพราะ path+mtime ตรง cache
Suppressed (Authenticode)	ไฟล์ที่ match แต่ถูก suppress เพราะ signature valid
Warnings (Unverified Pub)	ไฟล์ที่ match และ CompanyName อยู่ใน trusted list แต่ไม่มี signature valid
Suppressed (VT: 0 det.)	ไฟล์ที่ match แต่ VT malicious+suspicious = 0
Threats detected	จำนวน ALERT ที่ไม่ถูก suppress

14. Performance

- การตั้งค่านามสกุลไฟล์ช่วยลดเวลาในการสแกน
- เมื่อนำฐานข้อมูล signature เข้าในหน่วยความจำล่วงหน้าก่อนที่มีการสแกน
- การจำกัดไฟล์ 4KB–50MB ลด workload และหลีกเลี่ยงไฟล์ใหญ่เกินจำเป็น
- การใช้ worker pool ตามจำนวน CPU ช่วยให้คำนวณ ssdeep ได้ขนาน
- cache ลดเวลาสแกนซ้ำ โดยเฉพาะโพลเดอร์ขนาดใหญ่
- block size index ลดจำนวน signature candidate ที่ต้องเปรียบเทียบ
- การ query VT ทำเฉพาะเมื่อ ssdeep match แล้ว ไม่ได้ query ทุกไฟล์

15. ข้อจำกัดและความเสี่ยง

- ssdeep เป็น similarity matching จึงอาจมี false positive และ false negative
- cache ใช้ mtime เท่านั้น หากมี edge case ที่เนื้อหาเปลี่ยนแต่ mtime ถูกตั้งกลับ อาจข้ามการสแกน
- VT API มี rate limit และต้องพึ่งพา network
- CompanyName สามารถปลอมได้ จึงไม่ควรใช้เป็นหลักฐานว่าไฟล์ปลอดภัย
- บน Linux/macOS ไม่มี Authenticode และ CompanyName check จริง
- SQL import parser เป็น parser แบบง่าย อาจไม่ครอบคลุม SQL dump ที่ซับซ้อนหรือ string escape พิเศษ

16 สรุป ssdeep-scanner

เป็นเครื่องมือ Go CLI สำหรับสแกนไฟล์แบบ heuristic โดยใช้ ssdeep fuzzy hash เป็นแกนหลัก ผสาน SQLite, cache, worker pool และ VirusTotal เพื่อให้สแกนได้เร็วและลด false positive ในระดับหนึ่ง จุดแข็งของโปรเจกต์คือโครงสร้างตรงไปตรงมา build ง่าย และมี layer ตรวจสอบหลายชั้น ส่วนจุดที่ควรระวังคือความแม่นยำของ fuzzy hashing, ความน่าเชื่อถือของ CompanyName metadata, ความแม่นยำของโมเดลขึ้นอยู่กับคุณภาพข้อมูลและข้อจำกัดของ VT/API rate limit